
Speaker script

“Back to entropy” — evaluating language models as language models

Deck slides/talk.pdf — 13 slides + 4 backup slides

Target about 8:59 of talking at a relaxed 120 wpm (under 10 minutes), then Q&A

Goal The team leaves with one idea: judge a language model by the loss it was trained on

How to use this document. *Italic text is what to say — written to be spoken, not read verbatim. Orange brackets are delivery cues. Times are cumulative. The spoken length is checked by `python tools/script_timing.py`, which fails the build if it passes ten minutes.*

The spine — if only this lands, the talk worked:

1. Benchmarks are public and score a model before it is adapted.
2. Loss is the principled yardstick: it is the objective every model was trained on, and in bytes it is comparable across models.
3. Adapt every candidate the same way on fresh, decontaminated text, then measure bits per byte.
4. Adaptation reorders the models; the new ordering converges on HellaSwag, can be aimed at maths, is stable where benchmarks are not, and anticipates how fine-tuning will go.

SLIDE 1

0:00–0:18

Back to entropy

This talk is about one idea: that we should evaluate language models as language models. I will show why the usual way of choosing a base model misleads, what we do instead, and what we found.

SLIDE 2

0:18–1:11

How good is this model, for what we do?

Every week a new base model ships, and the question is always the same: how good is it, for what we do? A language model is trained to do exactly one thing, predict text. So the direct question is how well it predicts our text. Instead, we read a scorecard. Look at these two small models. On HellaSwag they are two points apart. On our news text, after adaptation, they are three percent apart. On GSM8K they are fifty-five points apart. Two models that model English almost identically well differ by nearly a factor of ten on one exam. Which of these numbers describes the model?

SLIDE 3

1:11–2:02

Two reasons the scorecard misleads

There are two reasons. First, benchmarks are public. They were written before the model existed, vendors optimise for the ones they report, and their content can end up in the training data. A high score might mean a stronger model, or a model that saw more of that exam, and nobody outside can tell which. Second, benchmarks score a model as it ships, but nobody deploys a base model as it ships. We adapt it first. Gemma-4-12B is ninth on GSM8K, eighth on MMLU-Pro, thirteenth of fifteen on raw loss — and once fine-tuned, it builds the best system of all fifteen.

[Land the Gemma example; it comes back on slide 10.]

SLIDE 4

2:02–2:59

Why loss is the right quantity

So what should we measure? There is no universal ground truth for which model is better, so we need a principled yardstick, and loss is the one. Every base model was trained to minimise cross-entropy. Held-out loss is its own objective, measured on text it has not seen. And cross-entropy splits cleanly: the entropy of the text, which is the same for every model, plus the model's distance from the text's true distribution. So a difference in loss is exactly a difference in how close each model is to the text. The only thing stopping a direct comparison is that each model counts in its own tokens. Divide by bytes and that goes away.

[LAND THIS: bits per byte is the training objective on a common scale.]

SLIDE 5

2:59–3:38

Taking that seriously: three requirements

Taking loss seriously asks three things. Bytes, so every tokenizer is on one scale. Fresh, decontaminated text, so no model has trained on it: we use text dated after every model shipped, we deduplicate and split with no overlap, and on the news snapshot we run three membership tests and remove anything any model flags. And equal adaptation: every candidate adapted to the domain by the same procedure, because we want to measure the model we will actually use.

SLIDE 6

3:38–4:34

Equal adaptation: the procedure, and what we ran

The procedure is a per-model Optuna search: a TPE sampler over learning rate, batch, rank and the rest, successive halving to prune weak trials, scored on the validation split, and every model gets the same number of trials. Then the winner trains until validation loss stops improving. That is the default in the released pipeline. Searching seventeen models up to 35 billion parameters was beyond our compute, so the results you will see use the pipeline's other mode: one hand-picked recipe for everyone, 250 steps — minutes per model. And note the trap at the bottom: if you cap the search by time, small models finish more trials and get tuned better.

SLIDE 7

4:34–5:19

What adaptation does to the ordering

Here is the result on one slide. On the left, fifteen models ranked five ways: by raw loss, by GSM8K, by MMLU-Pro, by adapted bits per byte, and by where they finish after we fine-tune them. Follow the blue line. Gemma-4-12B is thirteenth on raw loss and mid-table on both benchmarks. Once adapted it is second, and after fine-tuning it builds the best system of all. On the right, what the adapted ordering tracks: on general text it lands on HellaSwag, and on maths it swings to the reasoning benchmarks.

[LAND THIS: one picture, the whole argument.]

SLIDE 8

5:19–6:12

1. Adaptation reorders the models

First finding. Adaptation reorders the models: the rankings before and after agree only at about zero point six five. And it is not spread evenly. Adaptation cuts loss by four or five percent for every Qwen, Llama and Ministral model, but by seventeen to thirty-six percent for the two Gemmas and the Liquid model — and those three are the only ones that climb. The plot shows why: unadapted loss is mostly cold starts. Gemma opens each block far worse than a model eight times smaller and has passed it by the end. The unadapted number

measures familiarity with this kind of prose; adaptation removes that.

SLIDE 9

6:12–6:49

2. It converges on the language benchmark — and can be aimed

Second. HellaSwag is the benchmark closest to language modelling. Before adaptation our ranking correlates with it at about zero point seven; after, zero point nine six to zero point nine eight, on three different corpora. And adapt on mathematics instead, and the ranking swings toward GSM8K and MMLU-Pro. A benchmark is fixed; a corpus is chosen by whoever is choosing the model. So you can point this measurement at the capability you care about.

SLIDE 10

6:49–7:29

3. It is stable where benchmarks are not

Third. Seed-to-seed variation is about a thousandth of the spread between models, and not one of forty close pairs flips. Re-run weeks later on different hardware, cells match to the fourth decimal. Benchmarks do not behave like that: the same weights move GSM8K by more than half a point when only the host changes, and one decoding setting was worth ten points for Gemma. A generative benchmark measures the model, the harness, the decoding and the hardware at once.

SLIDE 11

7:29–8:14

4. Minutes of adaptation anticipate hours of fine-tuning

Fourth. Whatever model you choose, you will fine-tune it. So does the cheap number already tell you how that goes? We fine-tuned fifteen models on a news task. Adapted bits per byte, measured before the task existed, orders ninety-five percent of the pairs the way the fine-tune does. HellaSwag, eighty-seven. GSM8K and MMLU-Pro, a coin flip. Of course the two are related — both adapt the model to news — and that is the point: a few minutes of unlabelled adaptation already contain most of what hours of labelled fine-tuning reveal.

[LAND THIS: 0.949 against a coin flip for the reasoning benchmarks.]

SLIDE 12

8:14–8:52

Using it — and adding your model

In practice: if candidates are more than about two times apart in size, the bigger one usually wins. Otherwise, adapt every finalist the same way on recent domain text and rank by adapted bits per byte; treat anything under two percent as a tie. And the leaderboard is open: the maths track's corpus is public, so anyone can run a model and submit it, and the submission is checked against the exact corpus and recipe.

SLIDE 13

8:52–8:59

Evaluate language models as language models

So: evaluate language models as language models. Thank you — happy to take questions.

Q&A preparation

“Why not just fine-tune on the task?” When you have the task, its labels and the budget, do — it is the better evaluation for that task. A base model is usually chosen before any of those exist and has to serve

many tasks; the loss is what every one of them is built on.

“Doesn’t one hand-picked recipe favour some architectures?” It suits none perfectly. The full method searches each model; we could not afford it at 35B. A second, very different fine-tune recipe left the system ordering at Spearman 0.982.

“Does it beat HellaSwag?” It has the top point estimate everywhere and never falls below it, but at seventeen models the gap is not statistically resolved. We say so in the paper.

“What about contamination of our own text?” Three layers: dated after every release, deduplicated with no split overlap, and membership screens with a quarantine of anything any model flags.

“What does it cost?” With the hand-picked recipe, 3.7 GPU-minutes at 0.5B up to about an hour at 31B on one A100. The corpus is the main cost.